

Algoritmalar ve Veri Yapıları Bootcamp Ödev 1.

Çözümler

1. Öklid'in EBOB algoritması:

Algorithm *Euclid*(m, n)

//Verilen iki pozitif tam sayının EBOB'unu hesaplar.

//**Input:** İki adet negatif olmayan ve ikisi de birden 0 olmayan tamsayı: m ve n

//**Output:** m ve n 'nin EBOB'u

while $n \neq 0$ **do**

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

return m

Bu da, *ardışık tam sayı kontrol etme* esasına dayanan EBOB bulma algoritması:

Algorithm *GCD*(n, m):

// Verilen iki pozitif tam sayının EBOB'unu hesaplar

// **Input:** İki tamsayı, n ve m .

// **Output:** m ve n 'nin EBOB'u.

$d \leftarrow \min(n, m)$

while $d > 0$ **do**:

if $n \bmod d = 0$ **and** $m \bmod d = 0$:

return d

$d \leftarrow d - 1$

Öklid'in algoritmasına göre $\gcd(n, m) = \gcd(m, n \bmod m)$. Dolayısıyla;

$$\begin{aligned}\gcd(31415, 14142) &= \gcd(14142, 3131) \\ &= \gcd(3131, 1618) \\ &= \gcd(1618, 1513) \\ &= \gcd(1513, 105) \\ &= \gcd(105, 43) \\ &= \gcd(43, 19) \\ &= \gcd(19, 5) \\ &= \gcd(5, 4) \\ &= \gcd(4, 1) \\ &= \gcd(1, 0) \\ &= 1\end{aligned}$$

31415 ve 14142 sayılarının EBOB'u 1'dir. Öklid'in algoritması bunu bulabilmek için 10 iterasyon yaptı.

Ardışık tam sayıları kontrol eden algoritma için, algoritmanın her aday için her iki tam bölünebilirlik koşulunu da kontrol ettiği durum için (VE koşulunun her iki bileşeni de değerlendirilirse), 2x14142 bölme (mod bulma) işlemi yapmış olurduk (her iki sayıyı da bölmemiz gerekirdi). Bu örnek için Öklid algoritması bu örnekte *ardışık tam sayı kontrol etme esasına dayanan* algoritmaya göre 28284/10 = 2828,4 kat daha hızlıdır.

2.

Algorithm *ardisil_seri_bulma*(A[0.....n-1], k)

//Input: Bir tam sayı dizisi ve k değeri

//Output: Toplamı k olan ardışık elemanlar serisinin başlangıç indeksi.

Böyle bir seri yoksa -1.

for i ← 0 to n-1 **do**

 sum ← 0

 j ← i

while sum ≤ k **and** j < n **do**

 sum ← sum + A[j]

 j ← j + 1

if sum = k **then**

return i

return -1

Burada Problem boyutu açıkça dizinin boyutu, yani n'dir. En kötü durumda (hiçbir ardışıl serinin toplamı k'ye eşit değilse), algoritmanın hem dış hem de iç döngüleri tamamen tekrarlaması gerekir, çünkü algoritmanın erken bitip geri dönme olasılığı yoktur. Bu nedenle, analiz ifademiz (algoritmanın temel operasyonu karşılaştırma (while döngüsü için çalıştırılan koşul) olduğunu varsayarsak) şu şekildedir:

$$\begin{aligned} T(n) &= \sum_{i=1}^{n-1} \sum_{j=i}^{n-1} 1 \\ &= \sum_{i=1}^{n-1} (n - 1 - i + 1) \\ &= \sum_{i=1}^{n-1} (n - i) \\ &= (n-1) + (n-2) + (n-3) + \dots + 1 \end{aligned}$$

$$= ((n-1)n)/2$$

$$= (n^2 - n)/2$$

Dolayısıyla algoritmanın en kötü durum zaman karmaşıklığı **$O(n^2)$** 'dir.

Algoritmanın karşılaşılabileceği en iyi durumda ilk elemanın değerinin k'ya eşit olması, ya da ilk m adet elemanın (m<n) toplam değerinin k'ya eşit olması durumudur ki, böyle bir durumda dış for döngüsü sadece bir kez, iç while döngüsü de sabit sayıda, yani m kez döneceğinden (0<m<n); algoritmanın zaman karmaşıklığı **$O(1)$** yani sabit (problemin boyutundan bağımsız) olacaktır.